

PROJECT CONTEXT: SysAdmin Nexus (Tauri/Rust)

Role: Senior Frontend Architect & Rust Systems Engineer **Objective:** Implement the "SysAdmin Nexus" UI based on the attached high-fidelity design assets. **Tech Stack:**

- **Core:** Tauri v2 (Rust + Webview)
 - **Frontend:** React (TypeScript), Vite
 - **Styling:** TailwindCSS, Shadcn/UI (Radix Primitives)
 - **State:** TanStack Query + Zustand
 - **Charts:** Recharts (for Dashboard)
 - **Diagrams:** React Flow (for Automation)
 - **Terminals:** Xterm.js (for Remote)
-

1. Global Layout & Theme Strategy

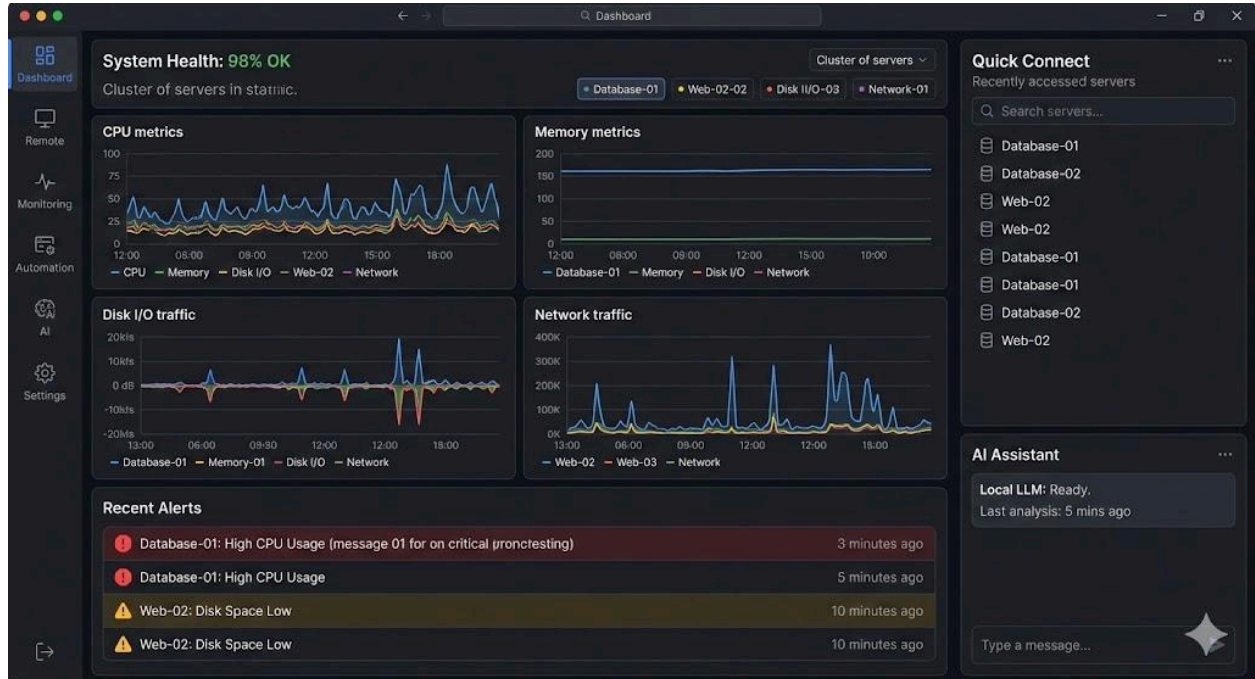
Reference Style: "Dark Mode First," high-density data, borderless window look.

- **App Shell:**
 - **Sidebar (Left, Fixed):** Navigation icons (Dashboard, Remote, Monitoring, Automation, AI, Settings). Active state highlights with a blue accent.
 - **Top Bar (Global):** Search bar (centered), Breadcrumbs (left), Window Controls (right).
 - **Main Content Area:** Scrollable, padded container **p-6**.
 - **Color Palette:**
 - Background: **#1a1b21** (Zinc-900 equivalent)
 - Card Background: **#27272a** (Zinc-800)
 - Accent: **#3b82f6** (Blue-500)
 - Alert Critical: **#ef4444** (Red-500)
 - Text: Inter or JetBrains Mono (for logs/terminals).
-

2. Module Implementation Specs

Module A: Main Dashboard ("Single Pane of Glass")

Visual Reference:



Component Tree:

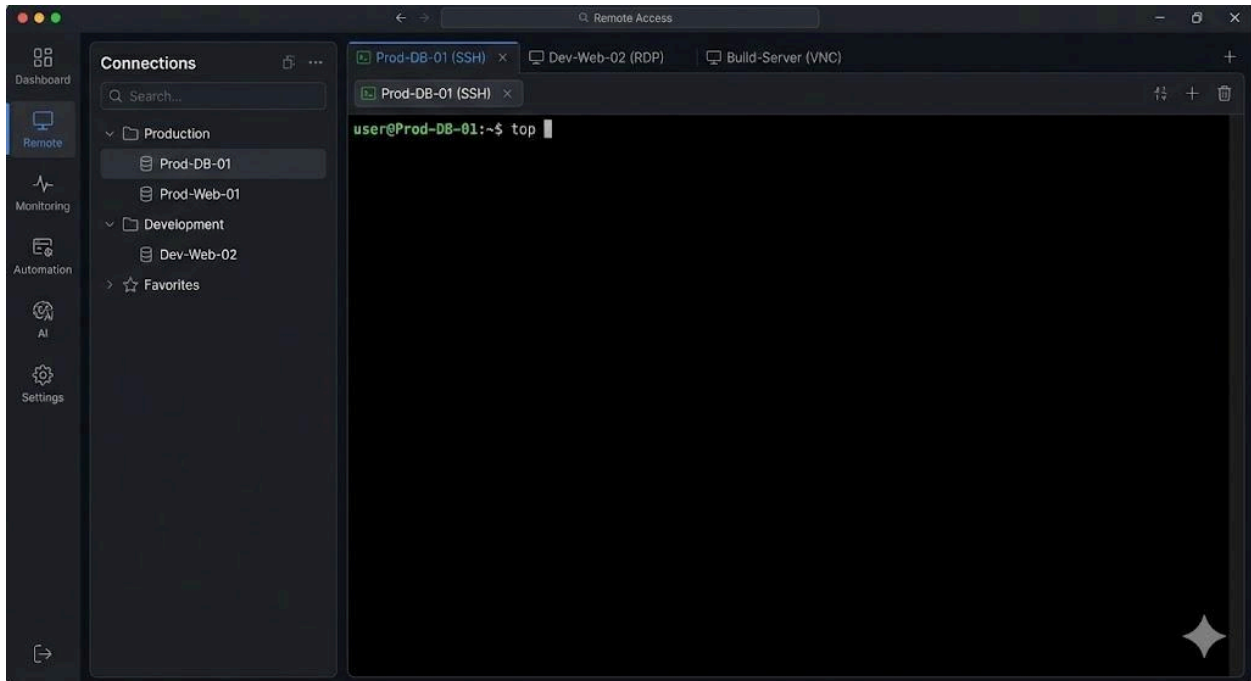
1. `DashboardLayout.tsx`: CSS Grid layout (3 columns).
2. `SystemHealthWidget.tsx`: Top banner.
 - *Props*: `healthScore` (98%), `statusText`.
 - *Visuals*: Green text for "OK", pills for cluster nodes.
3. `MetricChartCard.tsx` (Reusable):
 - *Library*: Recharts `ResponsiveContainer` -> `LineChart`.
 - *Data*: Real-time websocket feed from Rust backend (Glances/Netdata).
 - *Variants*: CPU, Memory, Disk I/O, Network.
4. `AlertFeed.tsx`: Bottom-left quadrant.
 - *Logic*: List item component with conditional border colors (Red/Yellow).
5. `QuickConnectSidebar.tsx`: Right column.
 - *Features*: Search input, list of recent connections (icon + label).

Agent Task:

"Scaffold the Dashboard grid. Use Recharts to mock the CPU/Memory graphs with smooth interpolation (`type="monotone"`). Implement the `AlertFeed` using a mapped array of mock alert objects."

Module B: Remote Connection Manager

Visual Reference:



Component Tree:

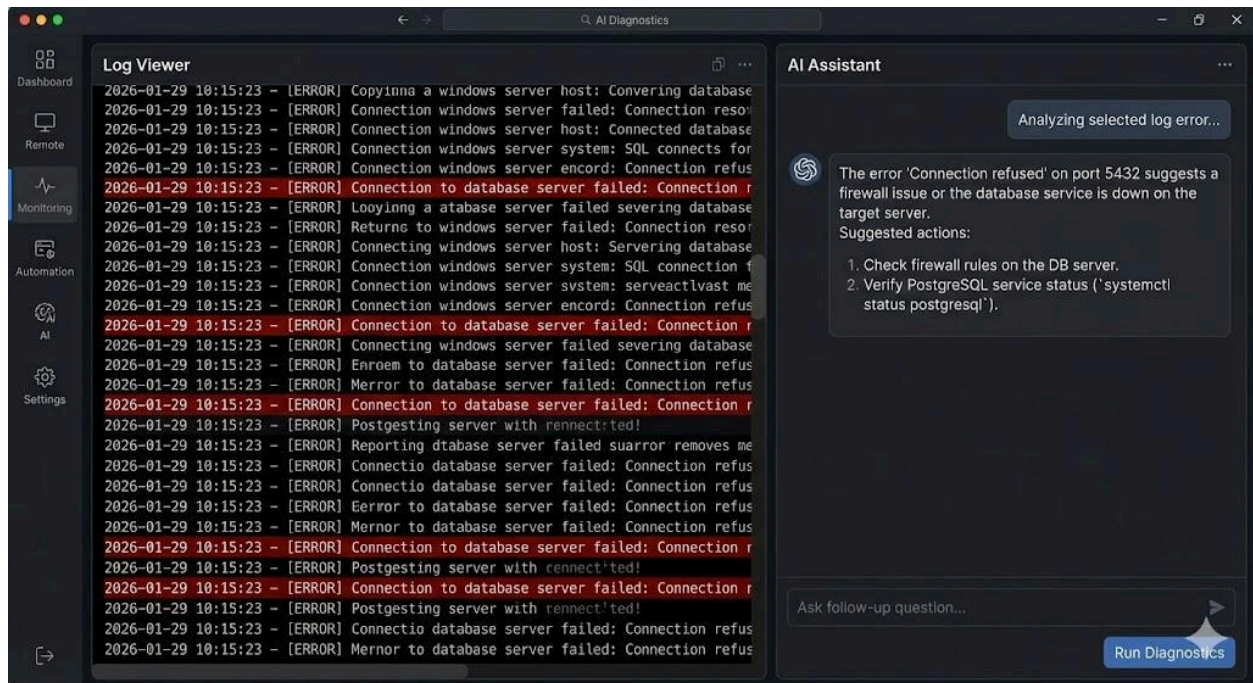
1. `ConnectionTree.tsx`: Left sidebar.
 - *Structure*: Nested folders (Production/Development).
 - *Interaction*: Click to open tab. Context menu for "Edit/Delete".
2. `TabManager.tsx`: Top bar.
 - *Behavior*: Chrome-style tabs. Active tab allows closing (x).
 - *State*: Manage `activeSessionId` and `openSessions[]`.
3. `SessionViewport.tsx`: Main area.
 - *Implementation*: Switch statement rendering sub-components based on protocol:
 - `SSHClient` (wraps `xterm.js`).
 - `RDPClient` (Canvas rendering stream).
 - `VNCCClient` (Canvas).

Agent Task:

"Create a `TabContext` provider to handle opening/closing remote sessions. Implement `ConnectionTree` using a recursive folder component. Integrate `xterm.js` into the `SessionViewport` and ensure it resizes correctly when the window changes."

Module C: AI Diagnostics Panel

Visual Reference:



Component Tree:

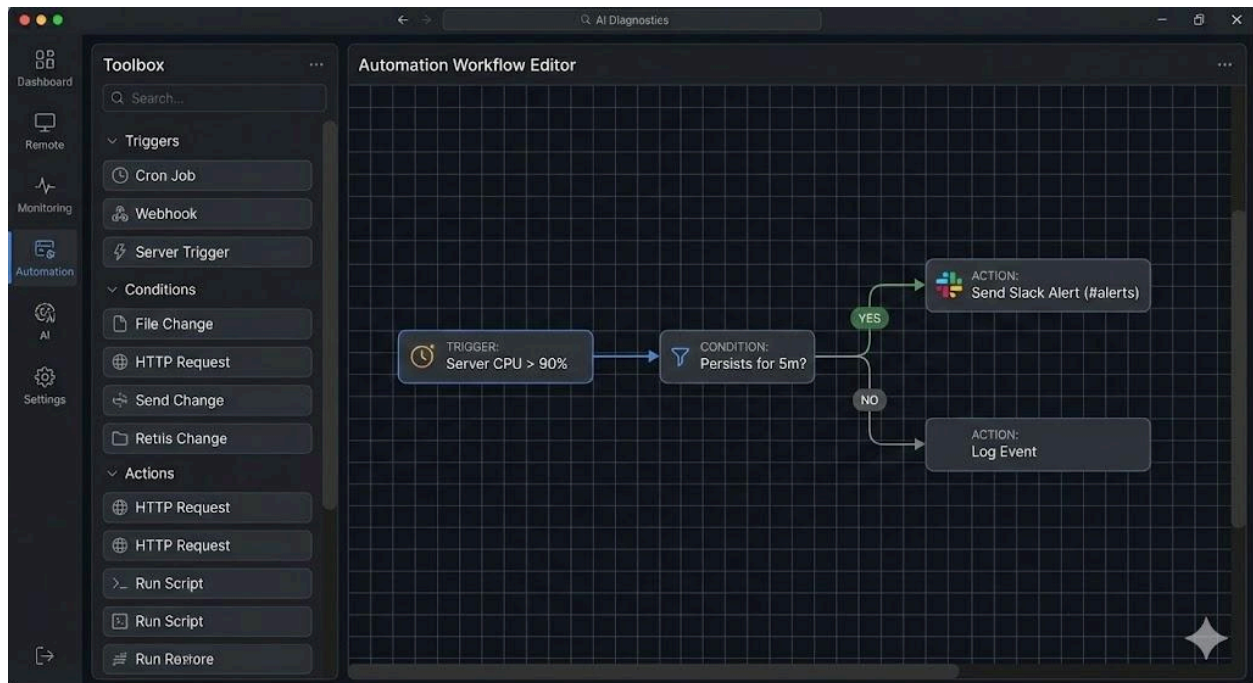
1. `SplitPaneLayout.tsx`: Resizable handle between Log Viewer and AI Chat.
2. `LogViewer.tsx`: Left pane.
 - *Features*: Virtualized list (React-Window) for performance with large logs.
 - *Styling*: Monospace font, distinct coloring for `[ERROR]` (Red) and `[INFO]` (Gray).
 - *Interaction*: Clicking a log line copies it to the AI input context.
3. `AIChatInterface.tsx`: Right pane.
 - *Components*: Message bubble (User vs AI), "Run Diagnostics" action button.
 - *Backend*: Connects to local Ollama instance via Rust sidecar.

Agent Task:

"Implement a split-pane view. The left side must render a mock log file with 1000+ lines efficiently. The right side should be a chat interface that accepts the selected log line as a 'system prompt' context."

Module D: Automation Workflow Editor

Visual Reference:



Component Tree:

1. **WorkflowCanvas.tsx:**
 - *Library:* React Flow.
 - *Config:* Dark mode background, grid pattern, mini-map.
2. **NodeToolbox.tsx:** Left sidebar.
 - *Items:* Draggable nodes for Triggers (Cron, Webhook), Conditions (If/Else), and Actions (Slack, Script).
3. **CustomNodes.tsx:**
 - *Design:* Card-like nodes with input/output handles.
 - *Trigger Node:* Blue border.
 - *Action Node:* Gray border.
 - *Logic Node:* Diamond shape or distinct header.

Agent Task:

"Initialize a React Flow instance. Create custom node types matching the design (Trigger, Condition, Action). Implement the Drag-and-Drop from the **NodeToolbox** to the canvas."

3. Implementation Roadmap (Step-by-Step)

Phase 1: The Shell (Days 1-2)

- **Objective:** Set up Tauri v2 + React + Tailwind.
- **Task:** Create the `Sidebar` and `TopBar` components. Ensure the layout is responsive and the "Dark Mode" theme variables match the hex codes provided.

Phase 2: Dashboard & Data (Days 3-5)

- **Objective:** Visualizing metrics.
- **Task:** Build the `MetricChartCard`. Mock data using a simple `setInterval` hook to ensure the charts animate smoothly. Build the `AlertFeed`.

Phase 3: Remote & Terminals (Days 6-10)

- **Objective:** SSH capability.
- **Task:** Integrate `xterm.js`. Create the `TabManager` state logic. Ensure multiple tabs can be "open" but hidden (display: none) to preserve session state when switching.

Phase 4: AI & Automation (Days 11-15)

- **Objective:** Advanced features.
- **Task:** Build the Split Pane for the AI module. Then, install React Flow and build the custom node styling for the Automation tab.

4. Coding Guidelines for Agent

1. **Strict Typing:** All components must use TypeScript interfaces.
2. **Performance:** Use `React.memo` for Chart and Terminal components to prevent unnecessary re-renders.
3. **Mock First:** Do not attempt to connect to real servers yet. Use `faker.js` or static JSON arrays for all data (Logs, Metrics, Inventory).
4. **Icons:** Use `lucide-react` for all iconography.